

Problem 8.3.3

$H_p = 0.93$, $H_s = 0.1$, $\omega_s = 3$ rad/s

Using find_n.py, $n = 3$

```
Hp = .93
Hs = .1
ws = 3
eps = sqrt((1/Hp**2) - 1)

aa = np.arccosh((1/eps)*sqrt((1/Hs**2)-1))
bb = 1 / np.arccosh(ws)
n = aa * bb

print('n = ', n)

n = 2.223014055460891
```

$\alpha = 5.25$, $a = 0.581$, $b = 1.16$ based on the code developed below

```
n = 3

alpha = 1/eps + sqrt(1 + 1/eps**2)
print('alpha = ', alpha)

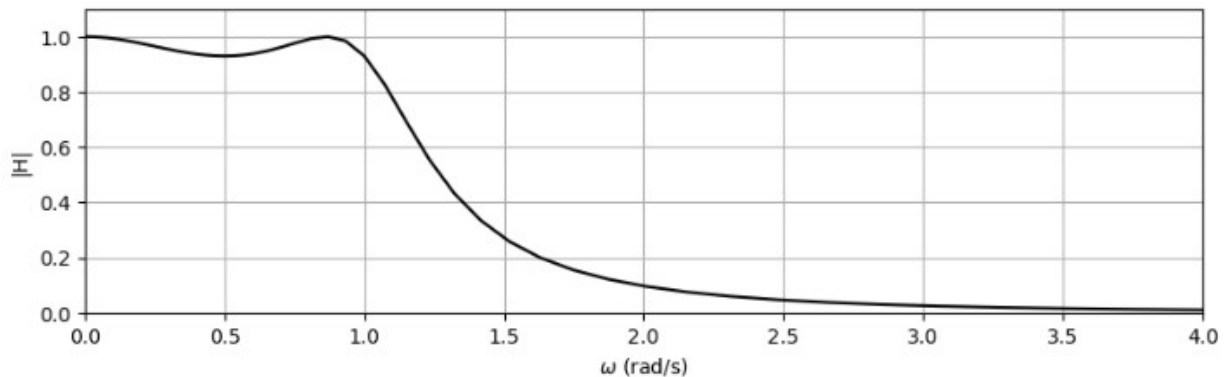
a = 0.5 * (alpha**(1/n) - alpha**(-1/n))
b = 0.5 * (alpha**(1/n) + alpha**(-1/n))
print('a = ', a, ' b = ', b)

theta = (180 - 60) * (pi/180)
d1 = [1, -2*a*cos(theta), (a**2)*cos(theta)**2 + (b**2)*sin(theta)**2]

d2 = [1, a]
K = d1[2] * a

num = [0, 0, K]
den = np.convolve(d1, d2)
```

This also calculated the transfer function, which gives the graph below



Problem 8.3.5

$$H_p = 10^{(-\frac{0.5}{20})} = 0.944, \quad H_s = 10^{(-\frac{20}{20})} = 0.1$$

Scale down w_p and w_s by dividing them by 1000,

$$w_p = \frac{1000}{1000} = 1, \quad w_s = \frac{200}{1000} = 0.2$$

and then the new w_s is

$$w_{s_{new}} = \frac{w_p}{w_{s_{old}}} = \frac{1}{0.2} = 5$$

Using these values with `find_n.py` gives $n = 2$

```
wp = 1
Hp = 10**(-0.5/20)
ws = 5
Hs = 10**(-20/20)

eps = sqrt((1/Hp**2) - 1)
aa = np.arccosh((1/eps)*sqrt((1/Hs**2)-1))
bb = 1 / np.arccosh(ws)
n = aa * bb
print('n = ', n)

n = 1.7632762536991264
```

$\alpha = 5.89$, $a = 1.00$, $b = 1.42$ based on the code developed below

```
n = 2

alpha = 1/eps + sqrt(1 + 1/eps**2)
print('alpha = ', alpha)

a = 0.5 * (alpha**(1/n) - alpha**(-1/n))
b = 0.5 * (alpha**(1/n) + alpha**(-1/n))
print('a = ', a, ' b = ', b)

theta = (180 - 45) * (pi/180) #90 for 2 poles
d1 = [1, -2*a*cos(theta), (a**2)*cos(theta)**2 + (b**2)*sin(theta)**2]

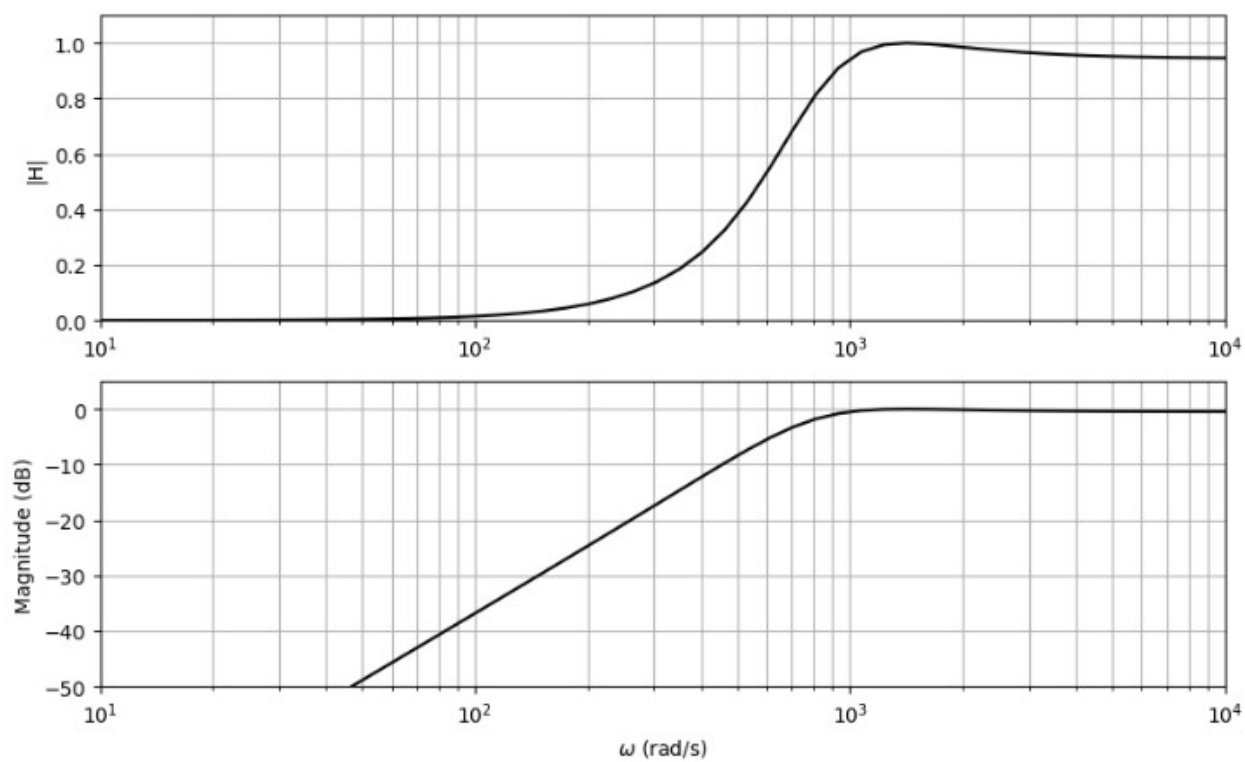
K = d1[2]*Hp

num = [0, 0, K]
den = d1

numh = num[::-1]
denh = den[::-1]

a = 1000
numh = [numh[0], a*numh[1], (a**2)*numh[2]]
denh = [denh[0], a*denh[1], (a**2)*denh[2]]
```

This also calculated the transfer function, which gives the graphs below



Problem 8.4.2

$H_p = 0.92$, $H_s = 0.1$, $\omega_p = 0.8$ rad/s

Using find_n.py, $n = 6$

```
Hp = .92
Hs = .1
wp = .8

eps = sqrt(Hs**2/(1-Hs**2))
aa = sqrt((1/eps**2)*(1/(1-Hp**2)))
bb = 1 / np.arccosh(1/wp)
n = np.arccosh(aa)*bb
print('n = ', n)

n = 5.665493332789188
```

The code below was used to calculate for the zeros and poles of the transfer function.

```
jj = complex(0,1)

alpha = 1/eps + sqrt(1 + 1/eps**2)
print('alpha = ', alpha)
a = 0.5*(alpha**(1/n)-alpha**(-1/n))
b = 0.5*(alpha**(1/n)+alpha**(-1/n))
print('a = ', a, ' b = ', b)

" Poles "
theta1 = np.deg2rad(15)
p1 = a*cos(theta) + jj*b*sin(theta)
q1 = 1/p1
d1 = [1, 2*q1.real, (q1.real**2 + q1.imag**2)]
print('d1 = ', d1)

theta2 = np.deg2rad(45)
p2 = a*cos(theta2) + jj*b*sin(theta2)
q2 = 1/p2
d2 = [1, 2*q2.real, (q2.real**2 + q2.imag**2)]
print('d2 = ', d2)

theta3 = np.deg2rad(75)
p3 = a*cos(theta3) + jj*b*sin(theta3)
q3 = 1/p3
d3 = [1, 2*q3.real, (q3.real**2 + q3.imag**2)]
print('d3 = ', d3)

d12 = np.convolve(d1, d2)
den = np.convolve(d12, d3)
```

```

" Zeros "
omega1 = 1/cos(1*np.pi/(12))
n1 = [0,0,0,0,1,0,omega1**2]
print('n1 =',n1)

omega2 = 1/cos(3*np.pi/(12))
n2 = [0,0,0,0,1,0,omega2**2]
print('n2 =',n2)

omega3 = 1/cos(5*np.pi/(12))
n3 = [0,0,0,0,1,0,omega3**2]
print('n3 =',n3)

n12 = np.convolve(n1,n2)
num = np.convolve(n12,n3)

```

This gives the results below.

```

alpha = 19.949874371066194
a = 0.5532488529694075 b = 1.1428404496306408
d1 = [1, 2.864851506995072, 2.680450677561488]
d2 = [1, 0.9706330190630458, 1.2405650479694768]
d3 = [1, 0.23112208387852828, 0.8070393228862727]
n1 = [0, 0, 0, 0, 1, 0, 1.0717967697244906]
n2 = [0, 0, 0, 0, 1, 0, 1.9999999999999996]
n3 = [0, 0, 0, 0, 1, 0, 14.928203230275514]

```

K is calculated when $H(0) = 1$, which gives the equation below

$$K = \frac{2.68 * 1.24 * 0.807}{1.07 * 2 * 14.9} = 0.0841$$

This makes the final transfer function

$$H(s) = \frac{0.0841(s^2 + 1.07)(s^2 + 2)(s^2 + 14.9)}{(s^2 + 2.86s + 2.68)(s^2 + 0.97s + 1.24)(s^2 + 0.231s + 0.807)}$$